X3 Capstone 3: review an AI-generated pull request

Generated from the same content files as the app. Each module lists its roadmap months. Solutions are in the appendix.

Contents

Track X: Capstones **2**
X3 Capstone 3: review an AI-generated pull request 3
X3-L1 Capstone 3: review an AI-generated pull request 3
Track X best-practice checklist 7
Appendix: solutions and answer keys **8**

Track X: Capstones

Three projects that combine the tracks, each checked against tests or a hidden key.

X3 Capstone 3: review an AI-generated pull request

Roadmap: Every month. Review a pull request an assistant wrote, graded against a hidden key.

X3-L1 Capstone 3: review an AI-generated pull request

Concept

This capstone brings Track D together. An assistant has opened pull request #214 against your pipeline repository. The full PR is in `practice/capstone/pr_review/PR.md`, with a review template beside it. The exercises below show the same five files, one per exercise, and a final verdict. The answer key stays hidden until you submit.

The PR's description says: "All tests pass. The code follows the team standard and has been tested thoroughly." Treat every sentence as a claim to check, not a fact (lesson D6-L1).

Review each file with the full routine:

1. **Goal.** What does this change say it does? The docstring or the description tells you.
2. **Read-through.** Names, types, side effects and scope (D6-L1).
3. **Known data.** Work one small case by hand. Does the code give your answer (D6-L2)?
4. **Assumptions.** What must be true of the data for this to work? Is it true (D6-L3)?
5. **Standard.** Nulls and truthiness, join grain, MERGE keys, tests that repeat the code, secrets (D3, D5, D7).

For every finding, record four things: the line, the defect, whether it blocks the merge, and the fix you request. A finding without a requested fix is a complaint, not a review.

Grading. Each file is a spot-the-bug exercise. Tapping the right line counts as a bug found. A fix that passes the checker counts as a fix proposed. The last exercise asks for your verdict and the reasons behind it.

Aim to find all five before you open a hint. Then compare your written review with the explanations. The defects missed most often in real reviews are the

ones that run cleanly: a truthiness test, an inner join, a MERGE key and a test that repeats the code.

Why it matters

More of the code you review will be written by assistants. It arrives with confident descriptions and passing tests. A reviewer who checks the claims, works an example by hand and reads the tests as critically as the code is what keeps those pull requests from quietly corrupting a pipeline.

Worked example: weak code

```
(1) Review: LGTM, tests pass.
```

1. **Line 1.** Repeats the author's claim. No finding, no check, no fix.

Worked example: strong code

```
1 Finding 1 (blocking): qa_statistical.py line 3 treats a 0.0 share as
    missing.
(1)  Checked: stability_status(0.0, 0.12) returns SKIP, expected WARN.
3  Fix: test `is not None` on both shares, and add a test for 0.0.
(2) Verdict: request changes. 4 blocking findings, 1 security finding.
```

1. **Line 2.** A worked example proves the finding.
2. **Line 4.** The verdict follows from the findings.

Every finding names the line, the defect, the severity, the check and the fix. The verdict follows from the findings.

Recap card

- **Five parts of a review finding** Line, defect, blocking or not, how you checked it, and the fix you request.
- **Defects this PR hid** Truthiness of 0.0, an inner join breaking grain, a MERGE missing wave_id, a self-confirming test, a token in code.
- **Treating a PR description** Every sentence is a claim. 'All tests pass' needs the tests to be read.

Practice

Exercise 1. Multiple choice (level 1)

PR #214's description says "All tests pass. The code follows the team standard and has been tested thoroughly." What do you do with these sentences?

- a) Treat each as a claim, and check it against the diff and the tests

- b) Accept them and review only the style
- c) Ignore the description entirely
- d) Approve, since the suite is green

Why?

- r1) A description states intent. Only the diff and the tests show what the code does.
- r2) Assistants always write false descriptions.

Exercise 2. Spot the bug (level 1)

File 1, `qa_statistical.py`. Goal from the docstring: WARN if the share moved more than the tolerance, SKIP only if a share is missing. Find the defect and fix it.

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN if the share moved more than tolerance, PASS if not,
    SKIP if missing."""
3     if current_pct and prior_pct:
4         return "WARN" if abs(current_pct - prior_pct) > tolerance else
    "PASS"
5     return "SKIP"
```

Write down the line number of the bug. Then write the fix.

Exercise 3. Spot the bug (level 2)

File 2, `gold_metrics.py`. Goal from the docstring: one row per gold row, with region names attached. Gold has region codes missing from the lookup. Find the defect and fix it.

```
1 from pyspark.sql import functions as F
2
3 def add_region_names(gold: DataFrame, regions: DataFrame) -> DataFrame:
4     """Attach region names. One row per gold row."""
5     return gold.join(regions, "region_code")
```

Write down the line number of the bug. Then write the fix.

Exercise 4. Spot the bug (level 3)

File 3, `silver_merge.sql`. Silver's grain is one row per respondent per wave. Find the defect and fix it.

```
1 MERGE INTO research.research_silver.nat_respondents AS t
2 USING research.research_bronze.nat_latest AS s
3 ON t.respondent_id = s.respondent_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;
```

Write down the line number of the bug. Then write the fix.

Exercise 5. Spot the bug (level 3)

File 4, `test_qa_statistical.py`. The description says "all tests pass". Find the line that makes this test worthless, and rewrite the test with hand-worked expected values, including the 0.0 case.

```
def test_stability_status():
    current, prior, tol = 0.30, 0.10, 0.10
    expected = "WARN" if abs(current - prior) > tol else "PASS"
    assert stability_status(current, prior, tol) == expected
```

Write down the line number of the bug. Then write the fix.

Exercise 6. Spot the bug (level 4)

File 5, `wave_api.py`. Find the security defect and fix it: read the token from the environment variable `SURVEY_API_TOKEN` at call time.

```
import requests

API_TOKEN = "tok_EXAMPLE_not_real_0000"

def fetch_wave(wave_id: str) -> dict[str, object]:
    """Fetch wave metadata from the survey platform."""
    headers = {"Authorization": f"Bearer {API_TOKEN}"}
    return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Write down the line number of the bug. Then write the fix.

Exercise 7. Architecture design (level 5)

Write the verdict section of your review.

Options: Request changes, Approve with comments, Approve.

1. Verdict
2. The description's 'all tests pass'
3. What you ask the author for

Sources

- Each hunk's behaviour follows from lessons verified earlier: truthiness of 0.0 (D3-L3), inner join drops unmatched rows (D3-L1), MERGE grain keys (A3, D4), self-confirming tests (D5-L3), secrets from the environment (D5-L3). (verified: Local runs recorded in the claims of the lessons named, September 2026.)

Track X best-practice checklist

☐ Read the brief and restate the goal before you touch code.

☐ Work to the CLAUDE.md standard, not to the starting code.

☐ Run the acceptance tests before and after each change.

☐ Record every Copilot or assistant output you rely on, with how you checked it.

☐ Compare your work with the reference or key only after you submit.

Appendix: solutions and answer keys

Attempt each exercise before you read its solution. Track D answer keys are here too.

X3-L1 Capstone 3: review an AI-generated pull request

X3-L1 exercise 1

Hints. (1) Lesson D6-L1: read before you trust. (2) What evidence backs each sentence? (3) Check each claim.

Answer. a) Treat each as a claim, and check it against the diff and the tests Because: r1) A description states intent. Only the diff and the tests show what the code does.

The description gives the goal. Each claim in it gets checked like any other assertion.

Common wrong answers.

- `b`: Style is the least of it. The claims are what you must check.
- `c`: The description tells you the goal, and its claims tell you what to verify.
- `d`: A green suite proves only what its tests check. Read them.
- `a|r2`: Right answer, wrong reason. Descriptions are often honest and still wrong about what the code does.

X3-L1 exercise 2

Hints. (1) Work stability_status(0.0, 0.12) by hand. (2) 0.0 is falsy. (3) `is not None` on both.

Answer. The bug is on line 3.

Fixed version:

```
1 def stability_status(current_pct: float | None, prior_pct: float | None,
    tolerance: float = 0.10) -> str:
2     """Return WARN if the share moved more than tolerance, PASS if not,
    SKIP if missing."""
3     if current_pct is not None and prior_pct is not None:
4         return "WARN" if abs(current_pct - prior_pct) > tolerance else
    "PASS"
5     return "SKIP"
```

Blocking. Reference defect 8 in a new form: the largest possible drop is never flagged.

Common wrong answers.

- Line 4: The comparison is right. The line above stops a 0.0 share reaching it.

X3-L1 exercise 3

Hints. (1) What happens to gold rows whose code is not in regions? (2) The default join type is inner. (3) Use a left join.

Answer. The bug is on line 5.

Fixed version:

```
1 from pyspark.sql import functions as F
2
3 def add_region_names(gold: DataFrame, regions: DataFrame) -> DataFrame:
4     """Attach region names. One row per gold row."""
5     return gold.join(regions, "region_code", "left")
```

Blocking. An inner join drops gold rows with unknown codes and breaks the grain the docstring promises. Also ask for a uniqueness check on the lookup, so a left join cannot duplicate rows.

Common wrong answers.

- Line 1: The import follows the standard.

X3-L1 exercise 4

Hints. (1) What is silver's grain? (2) Which key is missing from the ON clause? (3) wave_id.

Answer. The bug is on line 3.

Fixed version:

```
1 MERGE INTO research.research_silver.nat_respondents AS t
2 USING research.research_bronze.nat_latest AS s
3 ON t.respondent_id = s.respondent_id AND t.wave_id = s.wave_id
4 WHEN MATCHED THEN UPDATE SET *
5 WHEN NOT MATCHED THEN INSERT *;
```

Blocking. Matching on respondent alone overwrites a respondent's earlier waves with the new wave's row, and fails outright when a respondent has several waves in silver. Ask for source deduplication on both keys too.

Common wrong answers.

- Line 4: Updating matched rows is right, once the match uses the full key.

X3-L1 exercise 5

Hints. (1) Where does the expected value come from? (2) From the same formula as the code. (3) Write literal answers, including 0.0.

Answer. The bug is on line 3.

Fixed version:

```
1 def test_stability_status():
2     assert stability_status(0.30, 0.10, 0.10) == "WARN"
3     assert stability_status(0.12, 0.10, 0.10) == "PASS"
4     assert stability_status(0.0, 0.12, 0.10) == "WARN"
5     assert stability_status(None, 0.12, 0.10) == "SKIP"
```

Blocking. A self-confirming test (D5-L3) cannot catch the defect in file 1. The hand-worked 0.0 case fails against the current code, which is exactly what a test should do.

Common wrong answers.

- Line 4: The assert compares with `expected`. The problem is how `expected` was computed, one line up.

X3-L1 exercise 6

Hints. (1) Which line puts a credential into Git? (2) A literal token. (3) Read it from the environment at call time.

Answer. The bug is on line 3.

Fixed version:

```
1 import os
2
3 import requests
4
5 def fetch_wave(wave_id: str) -> dict[str, object]:
6     """Fetch wave metadata from the survey platform."""
7     headers = {"Authorization": f"Bearer
    {os.environ['SURVEY_API_TOKEN']}"}
8     return requests.get(f"https://survey.example/api/waves/{wave_id}",
    headers=headers, timeout=30).json()
```

Blocking, and a security finding. Even a placeholder invites someone to paste the real token. On Databricks, use a secret scope instead.

Common wrong answers.

- Line 8: The request has a timeout and reads the token from a variable. The variable is the problem.

X3-L1 exercise 7

Hints. (1) How many findings block the merge? (2) What does the only test actually check? (3) Targeted fixes or a restart?

Answer.

- Five findings, all blocking, one of them a security issue: request changes.
- A green suite made of a self-confirming test says nothing about the code.
- Ask for specific fixes and the tests that prove them, not a rewrite that starts the review over.

Request changes, name the unsupported claims, and ask for targeted fixes with hand-worked tests.

Common wrong answers.

- verdict = `approve_comments`: Comments are for non-blocking points. A wrong MERGE key and a committed token block the merge.
- verdict = `approve`: Five blocking defects, including a secret in code.
- claims = `supported`: The only test repeats the formula, so it passes whatever the formula does.
- ask = `rewrite`: A rewrite discards what is right and restarts the review. Ask for targeted fixes and tests.